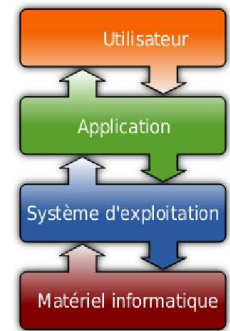


ARSE02**LES PROCESSUS****I. Les systèmes d'exploitation (OS)**

Souvent appelé l'OS (Operating System), le système d'exploitation est un ensemble de programmes qui permet d'utiliser les éléments physiques d'un ordinateur pour exécuter les applications nécessaires à l'utilisateur. L'élément fondamental du système d'exploitation est son noyau, c'est lui qui permet l'accès aux ressources matérielles.

Toute machine est dotée d'un OS qui a pour fonction :

- de charger les **programmes** depuis la mémoire de masse,
- de **gérer leur exécution** en leur créant des processus et en ordonnant les tâches,
- de **gérer les ressources** (microprocesseur, mémoire, disques, carte graphique, carte réseau, clavier, souris, ...)
- de **gérer des accès aux ressources** pour permettre : d'une part à tous les utilisateurs de travailler simultanément, et d'autre part de ne permettre l'utilisation d'une ressource qu'aux utilisateurs autorisés.
- d'**assurer la sécurité globale du système**.

**II. Les processus****Définition 1 : Les processus**

Un **processus** est une **instance** de programme en cours d'exécution sur un ordinateur. Il est caractérisé par :

- un ensemble d'instructions à exécuter souvent stockées dans un fichier sur lequel on clique pour lancer un programme (par exemple firefox.exe)
- un espace mémoire dédié à ce processus pour lui permettre de travailler sur des données et des ressources qui lui sont propres : si vous lancez deux instances de firefox, chacune travaillera indépendamment l'une de l'autre.
- des ressources matérielles : processeur, entrées-sorties (accès à internet en utilisant la connexion Wifi).

Un **processus** est un **programme** en cours d'exécution. Il ne faut donc pas confondre les deux. Il y a une différence entre programme et processus : le même programme peut être lancé plusieurs fois et donner naissance à des processus distincts.

Méthode 1 : Observation des processus

Sur chaque OS, il y a des utilitaires en mode graphique (GUI) ou en mode console (CLI) pour observer les processus en cours d'exécution.

- Sous Windows : gestionnaire des tâches (CTRL+ALT+SUPP).
- Sous Ubuntu : Moniteur systeme (Menu/Outils Systeme).

Sur les postes du lycée, l'accès au gestionnaire des tâches est verrouillé. Il est possible de créer un raccourci vers le terminal Windows Powershell avec un clic droit sur le bureau puis "nouveau raccourci" avec l'adresse : C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe. La commande ps permet de visualiser les processus comme dans le gestionnaire des tâches.

- Sous Linux, depuis le terminal, la commande pour afficher les processus est également ps (-e, -f) ou top. Utiliser l'émulateur du Terminal Linux en ligne JSLinux pour tester cette commande.

III. États des processus

L'OS doit permettre à toutes les applications et tous les utilisateurs de travailler en même temps, mais cette simultanéité n'est qu'illusion. En réalité, il va donner l'impression à chacun qu'il est seul à utiliser l'ordinateur et ses ressources physiques. Cela nécessite une gestion complexe des processus qui est réalisée par une partie spécifique du noyau du système d'exploitation, appelé : l'**ordonnanceur** (scheduler).

Un cœur de processeur ou un périphérique ne peuvent pas être partagés. Ce qui est réellement partagé c'est leur temps d'utilisation. L'ordonnanceur alloue la ressource à tour de rôle à chaque processus sur un intervalle de temps très court qui peut varier selon les besoins. Le plus petit intervalle de temps allouable est appelé le **quantum de temps** (20 à 50 ms suivant les OS et les algorithmes d'ordonnancement utilisés).

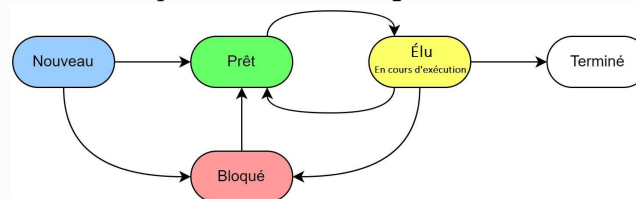
Les objectifs de l'ordonnanceur sont :

- de minimiser le temps de traitement du processus d'un utilisateur,
- de garantir l'équité entre les différents processus,
- d'optimiser l'utilisation des ressources,
- d'éviter les blocages.

Définition 1 : États des processus

C'est dans un registre (mémoire) que les instructions d'un processus en cours d'exécution sont copiées. Lorsqu'un processus est **interrompu** pour laisser la place à un autre processus, l'état des registres au moment de l'interruption doit être sauvegardé afin que lorsque ce sera de nouveau son tour d'être exécuté, son **contexte d'exécution** soit parfaitement rétabli et qu'il puisse poursuivre son exécution.

Tous les processus qui cohabitent en mémoire à un instant donné ne sont pas forcément en attente d'un quantum de temps processeur. Ils peuvent **attendre** d'avoir un accès à une autre **ressource** (périphérique d'entrée/sortie) ou bien ils peuvent également attendre le résultat d'un autre processus. Dans ces cas-là, on dit qu'il est dans l'état "**bloqué**". Au contraire quand un processus a tout ce qui lui faut et n'attend qu'un quantum de temps processeur, on dit qu'il est dans l'état "**prêt**".



🔧 Exercice 1 : Associer chacune des descriptions suivantes à l'état du processus correspondant :

1. Un processus a été attaché à une instance d'un programme et il n'a encore jamais été exécuté. Son contexte d'exécution initial doit être préparé.
2. La dernière instruction du processus a été exécutée. Ses ressources affectées et son environnement mémoire vont être libérés.
3. Un ou plusieurs quantum de temps processeur ont été alloués au processus. Son contexte d'exécution a été copié dans les registres et ses instructions sont exécutées jusqu'à l'écoulement du temps imparti.
4. Le processus est en attente d'une ou plusieurs ressources (réseau, périphériques d'entrée/sortie, action de l'utilisateur, ...) ou du résultat d'un autre processus. Il n'est pas éligible à un quantum de temps processeur.
5. Le contexte d'exécution est sauvegardé et complet. Le processus est en attente de quantum de temps processeur.

IV. Ordonnancement des processus

Définition 1 : Ordonnancement préemptif et non préemptif

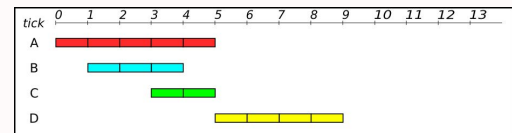
- un **ordonnancement non préemptif** est un ordonnancement où tout processus démarré n'a aucune raison de s'arrêter avant la fin. Si le système n'est pas préemptif, on peut penser à une organisation de type **file** (Premier arrivé, premier sorti : FIFO). (exemple : file d'attente d'impression).
- un **ordonnancement préemptif** est un ordonnancement où la priorité et donc l'élection peut être donnée à n'importe quel processus suivant un critère prédéfini. Si le système est préemptif, on réévalue le critère de priorité à chaque tour d'horloge. (exemple : prioriser les processus les plus courts, les plus longs, les plus rapides à se terminer, les plus *prioritaires*...)

Définition 2 : Quelques définitions

- Le **temps de séjour** d'un processus est la différence entre le temps de terminaison et le temps d'arrivée;
- Le **temps d'attente** est le temps de séjour auquel on retranche le temps d'exécution.

Méthode 1 : Dans la suite des exemples, nous prendrons les 4 processus suivants :

Imaginons que le processeur ait à exécuter 4 processus, dont les temps d'exécutions sont différents, et qui se sont présentés à différents instants au processeur :



1. Les systèmes non préemptifs

Méthode 2 : Méthode FCFS : First Come, First Serve

FCFS : First Come, First Serve. Les processus sont stockés dans une file dans l'ordre d'arrivée des processus.

1. Proposer le graphe d'exécution de cette méthode

2. Calculer le temps de séjour nécessaire à l'exécution de chaque processus
3. Calculer le temps de séjour moyen
4. Calculer le temps d'attente de chaque processus
5. Calculer le temps d'attente moyen

Méthode 3 : Méthode SJF : Short Job First

SJF : Shortest Job First. C'est difficile d'évaluer le temps d'exécution d'un processus mais c'est ce critère qui est utilisé pour prioriser leurs exécutions. C'est typiquement cet algorithme qui est appliqué aux caisses d'un supermarché, lorsqu'un client laisse passer un autre client ayant peu d'articles.

1. Proposer le graphe d'exécution de cette méthode

2. Calculer le temps de séjour nécessaire à l'exécution de chaque processus
3. Calculer le temps de séjour moyen
4. Calculer le temps d'attente de chaque processus
5. Calculer le temps d'attente moyen

2. Les systèmes préemptifs**Méthode 4 : Méthode SRT : Shortest Remaining Time**

SRT : Shortest Remaining Time. C'est difficile d'évaluer le temps d'exécution d'un processus mais c'est ce critère qui est utilisé pour prioriser leurs exécutions.

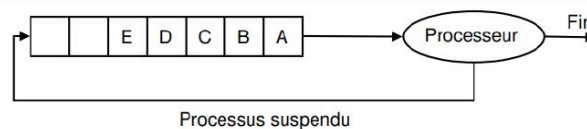
L'ordonnancement du plus petit temps de séjour ou Shortest Remaining Time (SRT) est la version préemptive de l'algorithme SJF. Un processus arrive dans la file de processus. L'ordonnanceur compare la valeur espérée pour ce processus contre la valeur du processus actuellement en exécution. Si le temps du nouveau processus est plus petit, il rentre en exécution immédiatement.

1. Proposer le graphe d'exécution de cette méthode

2. Calculer le temps de séjour nécessaire à l'exécution de chaque processus
3. Calculer le temps de séjour moyen
4. Calculer le temps d'attente de chaque processus
5. Calculer le temps d'attente moyen

Méthode 5 : Méthode Round - Robin

L'algorithme du tourniquet ou Round Robin (RR) représenté sur la figure ci-dessous est un algorithme ancien, simple, fiable et très utilisé. Il mémorise dans une file du type FIFO (First In First Out) la liste des processus prêts, c'est-à-dire en attente d'exécution.



1. si un nouveau processus est créé, il est mis dans la file d'attente;
2. ensuite on défile un processus de la file d'attente et on l'exécute pendant un quantum de temps;
3. si le processus exécuté n'est pas terminé, on le replace dans la file.

1. Proposer le graphe d'exécution de cette méthode pour un quantum de 1

2. Calculer le temps de séjour nécessaire à l'exécution de chaque processus
3. Calculer le temps de séjour moyen
4. Calculer le temps d'attente de chaque processus
5. Calculer le temps d'attente moyen

V. Interblocages

Les interblocages sont des situations de la vie quotidienne. Un exemple est celui du carrefour avec priorité à droite où chaque véhicule est bloqué car il doit laisser le passage au véhicule à sa droite.



Définition 1

Interblocages En informatique également, l'interblocage peut se produire lorsque des processus concurrents s'attendent mutuellement. Les processus bloqués dans cet état le sont définitivement. Ce scénario catastrophe peut se produire dans un environnement où des ressources sont partagées entre plusieurs processus et l'un d'entre eux détient indéfiniment une ressource nécessaire pour l'autre.

Exemple

Exemple d'interblocage

Considérons 2 processus A et B, et deux ressources R et S. L'action des processus A et B est décrite ci-dessous :

- A demande R et l'obtient
- B demande S et l'obtient
- A demande S bloqué par B. Il passe son tour
- B demande R bloqué par A. Il passe son tour
- ...

Processus A	Processus B
étape A1 : demande R	étape B1 : demande S
étape A2 : demande S	étape B2 : demande R
étape A3 : libère S	étape B3 : libère R
étape A4 : libère R	étape B4 : libère S

Les deux processus A et B sont donc dans l'état Bloqué, chacun en attente de la libération d'une ressource bloquée par l'autre : ils se bloquent mutuellement. Cette situation (critique) est appelée **interblocage** ou **deadlock**.

